

COURSE TITLE: CS6220 Big Data Systems and Analytics (Fall 2026, session cs6220-A)

HW1: Programming [X] Reading Critique []

Student Name: Atharva Mohite Student ID: 904308771

Live site (analysis write-up and interactive run explorer): <https://llm-prompting-lab.vercel.app/>

Project repository: <https://github.com/moriowen/llm-prompting-lab> (private)

Contents

Section	
Preface: how this document is organised	Structure, and the two populations of AI involved
Shared AI interaction record	Step 2 item 1 for all five questions: every AI tool used
Compute environment: PACE ICE	How the Georgia Tech A100 cluster was used, short and long
Question 1: model provenance and hyperparameters	Deliverable 1
Question 2: the result tables	Deliverable 2
Question 3: comparison across the LLMs	Deliverable 3
Question 4: comparison across the two tasks	Deliverable 4
Question 5: critique of the LLM reasoning outputs	Deliverable 5
Step 3: References	Does not count towards the page limit
Step 4: Appendices	Does not count towards the page limit

Preface: how this document is organised

The handout's Deliverable section poses five items, (1) through (5). This document treats each as one "question asked" and gives Step 1 and Step 2 for each. Step 3 (references) and Step 4 (appendices) come once at the end, as the template specifies.

The parts of Step 2 that are identical for every question, meaning the tool URLs and the overall multi-round strategy, are stated once in the block below. Each question then gives its own item 2 (exact prompts), 3 (round-by-round workflow), 4a (prompting strategy) and 4b (critique of the AI's response).

Shared AI interaction record (Step 2, item 1, for all five questions)

Two separate populations of AI were involved in this project, and keeping them apart is the methodological point of the whole submission. The **assistant** tools helped me design the experiment, write the harness and draft the analysis. The **subject** models are the seven models under test, which are the objects of study and the only source of the values in the result tables.

Tool	Population	What I used it for	Tool URL / download
Anthropic Claude (Opus 4.x / Opus 5) via the Claude Code CLI	Assistant	The bulk of the work: experiment design, the Python harness, the sweep and grading logic, the analysis, and this document. All Step 2 prompt records below are from these sessions.	https://claude.ai/code (CLI: <code>npm i -g @anthropic-ai/claude-code</code>); underlying API https://api.anthropic.com/v1/messages
OpenAI ChatGPT	Assistant	General question-and-answer and concept checking: understanding the assignment's framing, background reading on data contamination, and clarifying LLM concepts. Not used for the harness code, the grading, or the analysis.	https://chatgpt.com

Tool	Po pul ati on	What I used it for	Tool URL / download
OpenAI Codex	Assi sta nt	The presentation layer: the UI and report HTML, meaning the explorer page and report styling. Not used for the experiment code, the grading, or the statistics.	https://chatgpt.com/codex (CLI: <code>npm i -g @openai/codex</code>)
qwen2.5:1.5b, gemma3:4b, mistral:7b, qwen2.5:7b, llama3.1:8b, gemma3:12b, qwen3:32b	Sub ject	The objects of study. They produced only the answer strings inside the two result tables.	Served locally by Ollama, downloaded from https://ollama.com (https://ollama.com/download). Native endpoint <code>http://localhost:11434/api/chat</code> . Weights: <a href="https://ollama.com/library/<tag>">https://ollama.com/library/<tag> ; upstream cards on https://huggingface.co/

No assistant tool contributed a number to the tables, the grading, or the statistics. The division of labour matters for the critique in [Question 5](#): ChatGPT and Codex never touched the measurement path, so the instrument bugs described there belong to the Claude Code sessions and to me.

No hosted model API was used for the experiment itself. All 14,935 graded trials were decoded by a self-hosted Ollama server, either on my M2 MacBook Air (8 GB) or on a Georgia Tech PACE ICE A100 node reached through an SSH tunnel (`ssh -L 11434:<compute-node>:11434`), so the client always talked to `localhost:11434`. Each trace header records `backend: local-m2-air` or `backend: pace-ice-a100`.

The invariant I enforced on the assistant: no LLM output, from Claude or from the subject models, enters the ground truth, the grading, or the statistics. Queries and gold answers come from `src/generate.py` and a committed seed (6220). Task-2 answers are computed twice, once with `decimal.Decimal` under `ROUND_HALF_UP` and once with `fractions.Fraction`, and asserted equal. `main.py report` runs `verify.py` first and refuses to build if that assertion fails. The only AI-produced values anywhere in the deliverable are the cells of the result tables, which is the point of the assignment.

Compute environment: Georgia Tech PACE ICE

The short version

My laptop is an M2 MacBook Air with 8 GB of RAM, which caps me at roughly a 7B model at Q4 quantisation and cannot finish the CoT arms in reasonable time. So inference, and only inference, moved to a Georgia Tech PACE ICE A100. I connect the GT VPN, request a GPU node through Slurm, run an Ollama server on that node, and open an SSH tunnel that forwards the node's port 11434 to my Mac's own `localhost:11434`. Every other part of the project keeps running on the laptop. Because the tunnel makes the remote server appear at the same address a local one would, no code path changes between the two backends.

The longer version

The split is the point. Prompt building, grading, ground truth, traces and analysis all stay on the Mac. ICE serves nothing but Ollama.

Mac	ICE login node	ICE compute node
<code>main.py run --HTTP--> localhost:11434 --ssh -L--> :11434</code>		<code>ollama serve (A100)</code>
<code>prompts, grading,</code>		<code>gemma3:4b, mistral:7b, ...</code>
<code>runs/*.jsonl</code>		

The working sequence, which is recorded in full as `docs/PACE-ICE.md` in the repository:

1. Connect GlobalProtect to `vpn.gatech.edu`. PACE hostnames do not resolve off the GT network.
2. `ssh amohite8@login-ice.pace.gatech.edu`. The login node is for Slurm commands only.

3. `salloc -p ice-gpu --gres=gpu:a100:1 --cpus-per-task=8 --mem=64G --time=05:00:00`. The shell moves to a compute node whose name changes with every allocation, so nothing hardcodes it.
4. On the node: `module load ollama/0.12.11`, then `OLLAMA_HOST=0.0.0.0:11434 ollama serve > ~/ollama.log 2>&1 &`, then `ollama pull <tag>`. Binding `0.0.0.0` rather than loopback is what makes the node reachable through the tunnel.
5. From the Mac: `./scripts/ice-tunnel.sh`, which reads the current node out of `squeue`, opens `ssh -L 11434:<node>:11434`, and curls `/api/tags` through the forward so it reports `tunnel up: ... ollama answering` instead of failing silently.
6. From the Mac: `BDS_BACKEND=pace-ice-a100 python3 -m src.main run --models mistral-7b ...`.

Two design decisions follow from this arrangement and both matter for the results.

`BDS_BACKEND` is declared rather than inferred. A tunnel makes the two backends indistinguishable from the client, since both are `localhost:11434`, so the label is written into every trace header by hand. `wall_ms` from an A100 is not `wall_ms` from a fanless laptop, which is why accuracy is compared across backends in this report and timings are not.

Sampling settings do not change when the backend does. Everything in `src/utils/config.py` stays pinned exactly as it is, including `num_thread: 4`, which is inert on a GPU. Only the hardware moves, so the tables stay comparable. `gemma3:4b` was verified to have an identical digest (`a2af6cc3eb7fa8be850...`) on both machines before any run mixed them.

Four failure modes cost me time, and each reports something other than its actual cause. The login host is `login-ice.pace.gatech.edu`, not `pace-ice...`, and the wrong name produces `ssh: Could not resolve hostname`, which reads like a VPN failure. `srun --jobid=... --pty bash` hangs forever without `--overlap`, because PACE's `salloc` runs your shell as step 0 and holds the whole allocation. An interactive `salloc` job dies with the shell that launched it, taking the backgrounded `ollama serve` with it. And model stores are per node, so a new allocation usually means a node with nothing pulled. The last of these is why the incomplete Qwen3-32B Task-2 CoT cell exists: the tunnel dropped mid-run.

Question 1 (Deliverable 1): report each LLM with full name, version, company, parameter count, download URL and default hyperparameters

Step 1: Final answer (AI generated)

Seven open-weight, instruction-tuned models were run, all at Q4_K_M quantisation. The three in bold form the required three-model ladder, three organisations and three sizes, fixed before any data existed. The other four were added later, for reasons given in Question 3.

Alias	Full name (version)	Organisation	Parameters	Quant.	Ollama tag	Weights URL
qwen2.5-1.5b	Qwen2.5-1.5B-Instruct	Alibaba Cloud (Qwen team)	1.54 B	Q4_K_M	qwen2.5:1.5b	huggingface.co/Qwen/Qwen2.5-1.5B-Instruct
gemma3-4b	Gemma 3 4B Instruction-Tuned	Google DeepMind	4.30 B	Q4_K_M	gemma3:4b	huggingface.co/google/gemma-3-4b-it
mistral-7b	Mistral-7B-Instruct-v0.3	Mistral AI	7.25 B	Q4_K_M	mistral:7b	huggingface.co/mistralai/Mistral-7B-Instruct-v0.3
qwen2.5-7b	Qwen2.5-7B-Instruct	Alibaba Cloud (Qwen team)	7.62 B	Q4_K_M	qwen2.5:7b	huggingface.co/Qwen/Qwen2.5-7B-Instruct
llama3.1-8b	Llama 3.1 8B Instruct	Meta AI	8.03 B	Q4_K_M	llama3.1:8b	huggingface.co/meta-llama/Llama-3.1-8B-Instruct
gemma3-12b	Gemma 3 12B Instruction-Tuned	Google DeepMind	12.2 B	Q4_K_M	gemma3:12b	huggingface.co/google/gemma-3-12b-it

Alias	Full name (version)	Organisation	Params	Quant.	Ollama tag	Weights URL
qwen3-32b	Qwen3 32B	Alibaba Cloud (Qwen team)	32.8 B	Q4_K_M	qwen3:32b	huggingface.co/Qwen/Qwen3-32B

Every model is retrieved with `ollama pull <tag>` from `https://ollama.com/library/<tag>` and served over `POST http://localhost:11434/api/chat`. An Ollama tag is mutable and a digest is not, so `python3 -m src.main models` records the SHA-256 manifest digest of each pulled model into `data/models.json`. For example `qwen2.5:1.5b` resolves to `65ec06548149b04c096a120e4a6da9d4017ea809c91734ea5631e89f96ddc57b`. The run is reproducible against the digest, not the tag.

Default hyperparameters, and why I did not use them

Ollama's defaults are declared per Modelfile, so they differ between models. `gemma3:4b` ships `top_k 64`, `top_p 0.95`, `temperature 1`, `stop "<end_of_turn>"`, while `qwen2.5:1.5b` declares no parameter overrides at all and inherits Ollama's library defaults (`temperature 0.8`, `top_p 0.9`, `top_k 40`, `repeat_penalty 1.1`). Leaving these alone would have let two or three variables move at once across the size comparison. I pinned them identically for every model, arm and temperature, and report the per-model defaults separately (captured by `main.py models`) as this deliverable requires.

Setting	Value used everywhere	Why pinned
temperature	the swept variable, 0.0 to 1.0 in 0.1 steps (11 points)	the independent variable
top_p	1.0	disables nucleus truncation so temperature acts alone
top_k	40	Gemma defaults to 64; unpinned it would confound size with sampling
repeat_penalty	1.1	same reason
num_ctx	2048	fits every prompt including 3-shot verbose
num_thread	4	laptop-safe, irrelevant on the A100
seed	per-draw, recorded per trial	makes each draw reproducible
num_predict	keyed by (task, arm): 128 for zero-shot and few-shot, 640 for Task-1 CoT, 1536 for Task-2 CoT	stated deviation, see Question 5
stop	<code>["\n"]</code> for the single-answer arms, none for CoT	stated deviation, see Question 5

Step 2: AI interaction record

- 1) Tool URL.** See the shared block above. Claude Code (`https://claude.ai/code`) for the assistant, Ollama (`https://ollama.com`) for all seven subject models.
- 2) Exact prompts used.** Verbatim from my terminal history, typos included:

use the programming assignment PDF to extract the understanding of what needs to be done in a README
 can you critique the analysis. Also suggest what models can be used for this use case. I plan to make a HTML report along
 with a proper repo for this project. I want to follow the structure like the db agent project, but should I use langchain
 rather than actually writing the harness for this use case? path - /Users/atharvamohite/dev/dbagent. let's brainstorm, create
 a new md file for this

two tasks for you: 1. I don't trust the h1.md and @H1-analysis.md because it was created by a non-thinking model. Go through
 the @2026-ProgrammingAssignment-1.pdf and make your own conclusions. Update the @H1-critique-and-build.md with all the
 understandings and remove all the rest of the md files. Make sure there is one more md that is just the transcript of what
 the PDF says. The other md should be your analysis of the problem and plan of action. 2 - only want to run self-hosted
 models for this. so any setup/brainstorming related to using hosted models should be discouraged. Although I have an M2 air
 with 8GB RAM only, so I can't run big models, can you accommodate for that as well

ok on the sweep works with 5. I am a little concerned whether the 7B model will run on my machine. For now I just want to run
 it with 1B and then progressively increase from there. Write the code for it, but don't download the models. I already have
 Qwen 2.5 1.5B parameter model downloaded on my local, so I want that to be the first one. You can use whatever other models
 you can get for 4 and 7.

are there any other reliable and recent 7B models?

let's do qwen7b, llama7b and gemma 12b

can I run qwen 32B and 70B models on A100 cluster?

let's download and run qwen3 32b

3) Multi-round workflow, with the context and query used in each round.

Round	Context supplied	Query	Outcome
1	The assignment PDF	extract requirements into a README	First pass. I later rejected it.
2	Prior analysis .md files plus a reference repo path	critique the analysis, suggest models, LangChain or a hand-written harness?	Claude proposed a hosted-API plus LangChain design. Rejected.
3	Same PDF, plus my two constraints	re-derive everything yourself, self- hosted only, 8 GB M2 Air	Produced H1-assignment-transcript.md (verbatim handout) and H1- critique-and-build.md (the plan). Model ladder chosen under the 8 GB ceiling.
4	The ladder proposal	pin to Qwen2.5-1.5B first, scale up later, download nothing yet	MODEL_REGISTRY written with installed: false flags.
5	PACE ICE access working	which recent 7B models are reliable? then: run qwen7b, llama8b, gemma 12b	Registry extended to six.
6	A100 40 GB capacity	can I run 32B or 70B here? then: download and run qwen3 32b	32B added. Its 19.9 GB of Q4 weights fit; 70B does not.

4a) Prompting strategy and multi-round AI-use strategy.

My strategy for the assistant had three deliberate features.

The first was to distrust the first draft and force re-derivation. Round 2's answer was produced from summaries of the problem rather than the problem itself. Round 3's prompt, "I don't trust the h1.md ... make your own conclusions", required Claude to read the PDF and emit two separate files: a verbatim transcript and an analysis. Separating source from interpretation meant later rounds could be checked against the handout's own words rather than a paraphrase that might have drifted.

The second was to state constraints as hard boundaries. "Self-hosted only ... hosted models should be discouraged" and "M2 air with 8GB RAM" were given as vetoes. That turned model selection from an open question into a search inside a real feasible set, and it is why `mistral:7b` (4.1 GB) rather than `qwen2.5:7b` (4.7 GB) took the original large slot.

The third was to make provenance machine-checked instead of transcribed. I did not let Claude type the model table by hand. `main.py models` queries the local Ollama install for digests and `ollama show --parameters` output, and the report table is generated from `MODEL_REGISTRY` plus `data/models.json`. A hand-written table goes stale the moment a tag is re-pulled.

4b) Critique of the AI's response.

What it got right: the size and memory arithmetic was correct and useful. Claude identified that Q4_K_M weights run at roughly 0.55 bytes per parameter plus KV cache, correctly ruled out a 70B on a 40 GB A100, and flagged that Ollama's per-Modelfile defaults would confound the size comparison if left alone. That last point is not obvious and it changed the design.

What it got wrong: round 2 proposed a LangChain-mediated, hosted-API design. That is the default answer to "how do I call several LLMs", and it was wrong here in three ways Claude did not raise until challenged. It violates the self-hosting constraint. LangChain's abstraction hides `num_ctx`, `top_k` and `seed`, which are the hyperparameters this assignment asks me to control and report. And it adds a dependency whose version drift is itself an uncontrolled variable. The workable answer, `urllib` against Ollama's native `/api/chat` with zero dependencies, was reachable only after I supplied the constraint. The assistant optimised for convenience where the assignment rewards control.

A subtler oversight: Claude's first registry proposed exactly three models, matching the handout's minimum. That would have been defensible, and as Question 3 shows it would have produced a nearly all-zero Task-2 table with no visible relationship between size and accuracy. The threshold I eventually found sits between 7.25 B and 7.62 B, inside the gap the three-model ladder leaves unsampled. The AI satisfied the stated requirement without asking whether the requirement would produce an informative result.

A limitation still standing: `data/models.json` carries captured digests only for the two models ever pulled to the laptop. The five pulled on the PACE node have `digest: null` locally. That is a real gap in my provenance record, and those five are pinned by tag rather than by digest.

Question 2 (Deliverable 2): result tables for both tasks, 10 queries by 3 LLMs by 2 hyperparameter settings, wrong answers in red

Step 1: Final answer (AI generated)

The two hyperparameter settings are `t = 0.0` (greedy) and `t = 1.0`. They were not chosen by eye. `config.INFLECTION_CURVE = "accuracy"` and the selection rule were written into code before the data existed: sweep all 11 temperatures, look for a knee (a single-step drop of 0.15 or more), and if the whole curve's range is under 0.10, declare it flat and take the endpoints. The observed pooled range was 0.03, so `sweep.py` returned `shape: "flat_at_floor", knee: null, pair: [0.0, 1.0]`. Cells report the modal normalised answer over the draws at that temperature.

Wrong answers are shown in bold in this markdown rendering. In the submitted Word or PDF version every bold cell is set in red font, as required. Correctness is assigned by `src/grade.py` against Python-computed gold, never by hand.

Table 1: Character-By-Character Reversal (required three-model ladder)

Query	Qwen2.5-1.5B t=0.0	Qwen2.5-1.5B t=1.0	Gemma-3-4B t=0.0	Gemma-3-4B t=1.0	Mistral-7B t=0.0	Mistral-7B t=1.0	Gold
Q1: reverse PkUuq (l=5)	rqsw	rqWk	qUuqP	qUuqP	qqqUuK	qqUkP	quUkP
Q2: reverse inNjf (l=5)	fjnil	fjen	jfNni	jfNni	ffJni	fJngi	fjNni
Q3: reverse fugJr (l=5)	rgsuf	gufj	jrguF	jrguF	gurJf	jurFg	rJguf
Q4: reverse JQJPc (l=5)	PCJQ	PCJQ	cPJQJ	cPJQJ	pcJQJ	pcJQJ	cPJQJ
Q5: reverse gypHq (l=5)	rqygp	rqyp	qHpyg	qHpyg	qopHyg	qpHyg	qHpyg
Q6: reverse c1BFMztw (l=8)	mtswzFMBlc	zwyfGMcl	twzBMFBC	wztBMfLC	wtzFMBlc	wtzoBmFcl	wtzMFBlc
Q7: reverse mbwCcDd0 (l=8)	dcd0CmB	dcDOcccBm	odDOcCCwbm	odO DdCcwbm	doODwcM	DoOcCbwm	OdDcCwb m

Query	Qwen2.5-1.5B t=0.0	Qwen2.5-1.5B t=1.0	Gemma-3-4B t=0.0	Gemma-3-4B t=1.0	Mistral-7B t=0.0	Mistral-7B t=1.0	Gold
Q8: reverse uVTztcSp (l=8)	czspTuvu	ctspuTV	pStczVtU	pStczVtU	pSpctczVu	pHwLmDqoN	pSctzTVu
Q9: reverse VAYCXPhA (l=8)	XCAPHYV	ECAPB	AHXPCYAV	AHXPCYAV	ahCXPyav	AhabacXyPV	AhPXCYAV
Q10: reverse EklaSuVr (l=8)	lruS	lkresuEk	VrlaSuKe	VrlaSuKe	VruSulEk	vRuSvLake	rVuSalkE
correct / 10	0	0	2	2	0	0	

Table 2: Decimal Computation with Rounding (required three-model ladder)

Query	Qwen2.5-1.5B t=0.0	Qwen2.5-1.5B t=1.0	Gemma-3-4B t=0.0	Gemma-3-4B t=1.0	Mistral-7B t=0.0	Mistral-7B t=1.0	Gold
Q1: 744 / 336 @ 5 dp	2.21805	2.20189	2.20535	2.20695	2.2187820	744	2.21429
Q2: 944 / 960 @ 5 dp	0.98137	0.97231	9.33333	9.33333	0.97872346	944	0.98333
Q3: 159 / 512 @ 5 dp	0.31064	0.31064	3.14141	3	0.3078054	159	0.31055
Q4: 903 / 672 @ 5 dp	1.40833	1.29846	1.33335	1.33335	1.34680852	5	1.34375
Q5: 364 / 710 @ 5 dp	0.50982	0.51298	3	3	0.5098048	5	0.51268
Q6: 258 / 243 @ 8 dp	10.76923077	10.67972604	1.0610610610610 61	1.0610610610610 61	1.061702609375	258	1.061728 40
Q7: 273 / 861 @ 8 dp	0.31459459	0.03194509	2.9739409683940 97	2.9716829717	0.31578947368421 05	0.3145873411	0.317073 17
Q8: 81 / 768 @ 8 dp	0.10734259	0.10638292	1.0603947368421 05	1.0603947368421 05	0.10526315789473 684	81	0.105468 75
Q9: 525 / 256 @ 8 dp	2.07142857	20.57142857	2.048330078125	2.0483818381838 183	2.07916666747058 83	2.04761904761 905	2.050781 25
Q10: 165 / 531 @ 8 dp	0.30724946	0.309274075	3.0415929203041 592	3.0415929203041 592	0.310888344726	0.31088834472 664	0.310734 46
correct / 10	0	0	0	0	0	0	

Tables 3 and 4: the same two tables over the three larger models

Table 2 is entirely red. That is a result worth reporting, but a table with no variance cannot support deliverables (3) and (4), which ask me to compare across models. So I ran three larger models and repeat both tables over them. These are supplementary. The required ladder is above.

Table 3: reversal, larger models

Query	Llama-3.1-8B t=0.0	Llama-3.1-8B t=1.0	Gemma-3-12B t=0.0	Gemma-3-12B t=1.0	Qwen3-32B t=0.0	Qwen3-32B t=1.0	Gold
Q1 PkUuq	qUUKp	uquUKp	quuPk	quuPk	quUPk	quUPk	quUkP
Q2 inNj f	fjnNi	fojNi	fjNni	fjNni	fjNni	fjNni	fjNni
Q3 fugJr	rojGuf	rgJuF	Jrgu	Jrgu	rJguF	rJuGf	rJguf
Q4 JQJPc	pcPJQ	pciPJq	cPJQJ	cPJQJ	cPJQJ	cPJQJ	cPJQJ
Q5 gypHq	qHpYg	aqpHyg	qypHg	qpHyg	qHyPg	qHp yg	qHpyg
Q6 cLBFMztw	wtzMFBlc	wtmuzFBlc	wzMtFBlc	wztMFBlc	wtzMFBlc	wtzMFBlc	wtzMFBlc
Q7 mbwCcDdO	OdCddBwm	oodDaCwcBm	OddCcbm	OdddCcbm	OdDeCwbm	OdDeCwbm	OdDeCwb m
Q8 uVTztcSp	pSctzTVu	pscTztVu	psTcZtVu	pszcTvu	pScztTVu	pSctzTVu	pSctzTVu
Q9 VAYCXPhA	AHCPYAV	aHpCYAV	AhPxCYAV	AhpxCYAV	AHPXCYAV	AHPXCYAV	AhPXCYAV

Query	Llama-3.1-8B t=0.0	Llama-3.1-8B t=1.0	Gemma-3-12B t=0.0	Gemma-3-12B t=1.0	Qwen3-32B t=0.0	Qwen3-32B t=1.0	Gold
Q10 EklaSuVr	orvVuLasKe	rivUslAkE	VrSuEkla	VrSuEkla	rVUklaSkeL	rVUkaLkE	rVuSalkE
correct / 10	2	0	2	2	4	5	

Table 4: division, larger models

Query	Llama-3.1-8B t=0.0	Llama-3.1-8B t=1.0	Gemma-3-12B t=0.0	Gemma-3-12B t=1.0	Qwen3-32B t=0.0	Qwen3-32B t=1.0	Gold
Q1 744/336 @5	1.11389	2.21071	2.21667	2.21667	2.21429	2.21429	2.21429
Q2 944/960 @5	0.98333	0.98298	0.98333	0.98333	0.98333	0.98333	0.98333
Q3 159/512 @5	0.31094	0.31042	0.31039	0.31133	0.30996	0.30996	0.31055
Q4 903/672 @5	0.13458	0.13460	1.34551	1.34242	1.34375	1.34375	1.34375
Q5 364/710 @5	0.51148	0.51261	0.51268	0.51268	0.51268	0.51268	0.51268
Q6 258/243 @8	2.106383	0.106067012	1.06194174	1.06194174	1.06172414	1.06172840	1.06172840
Q7 273/861 @8	0.31665493	0.31701361	0.31721311	0.31734346	0.31707317	0.31707317	0.31707317
Q8 81/768 @8	0.105208	0.10520109	0.01052632	0.01052632	0.10546875	0.10546875	0.10546875
Q9 525/256 @8	0.204668359	0.204294687	2.05078125	2.05078125	2.05078125	2.05078125	2.05078125
Q10 165/531 @8	0.31023563	0.31045365	0.31071424	0.31071424	0.31073446	0.31073446	0.31073446
correct / 10	1	0	3	3	8	9	

All four tables come from the zero-shot arm, so the columns differ only in model and temperature.

Step 2: AI interaction record

1) Tool URL. As per the shared block.

2) Exact prompts used.

```
For the design constraint, I want to run N = 10, which goes from 0 to 1 with a 10-point increment. That way, I can pick the values exactly where there is a point of inflection, so that I can reliably point to where the model fails. For the source of truth data set, I want to use a Python script that will do all of the computation and do not want to rely on the model output. The different types of prompting techniques covered in class are zero-shot, few-shot, and chain-of-thought prompting, so I would want to experiment with all three of them. update the plan of action with these considerations
two tasks: create a minimalistic ui to visualize the results of the runs and can you run this experiment with the 4B model as well
what does seed mean here? and why does qwen not have temp wise runs like gemma?
what is accuracy and consistency in the graph plotted?
can we run task2 zero and few shot for all models (not verbose prompts)
also every query is being sent to the model 5 times, change it to 3, we need faster runs
remove the normalized and strict grading toggle, just use the normalized value
```

3) Multi-round workflow.

Round	Context	Query	Outcome
1	The plan document	N=10 sweep 0 to 1, ground truth in pure Python, all three class prompting techniques	Froze TEMP_GRID, the two-method Task-2 verification, and the three arms.
2	Working harness, 1.5 B runs	build a minimal UI, add the 4 B model	src/ui.py, a self-contained explorer with no server and no CDN.
3	First curves	what does seed mean here? why does Qwen lack per-temperature runs?	Exposed a real gap: the 1.5 B sweep had not been run across the grid.

Round	Context	Query	Outcome
4	Plotted curves	what are accuracy and consistency in this graph?	Forced Claude to define its own metrics. Self-consistency was demoted to corroboration and accuracy fixed as the selector.
5	7 models registered	run Task 2 zero-shot and few-shot for all models, not the verbose arms	Completed the grid used in Tables 2 and 4.
6	Throughput problem	k = 5 down to 3	Cut wall-clock. k = 10 retained only at the two reported temperatures.
7	Report draft	drop the strict/normalised toggle, report normalised only	One headline number. Strict is retained internally to separate formatting from arithmetic errors.

4a) Prompting strategy and multi-round AI-use strategy.

The decision that makes these tables worth anything was made in round 1: the selection rule for the two hyperparameter settings was written into code before the data existed. I asked for an 11-point sweep so the reported pair could be chosen at a measured inflection point instead of asserted. When the sweep came back flat with a range of 0.03, the same pre-committed rule returned the endpoints. Had I chosen after seeing the data, "0.0 and 1.0" would be indistinguishable from a convenient default. Because the rule was frozen, the flatness is itself the reported result.

The second decision was to forbid the assistant from touching ground truth: "I want to use a Python script that will do all of the computation and do not want to rely on the model output." Claude wrote `generate.py` and `verify.py`, but they compute rather than consult a model. Task-2 gold is computed twice by independent means and asserted equal, and the report build refuses to run if the assertion fails.

Rounds 3 and 4 are the ones I would point to as intelligent use. Both are interrogations rather than instructions: "why does Qwen not have temperature-wise runs like Gemma?" and "what is accuracy and consistency in the graph you plotted?" Asking an assistant to define the quantity it just plotted is cheap and it caught two problems, a missing sweep and a metric (self-consistency) that was drifting towards being used as a selector when it does not measure correctness at all. A model will happily plot a metric it has not defined.

4b) Critique of the AI's response.

The harness is correct where it matters. Modal-answer selection per cell, resumability keyed on `(qid, seed)`, per-line flushing so a thermally throttled laptop still leaves an analysable prefix, and the disjointness assertion between the few-shot pool and the test set are all sound. The last of these is a contamination control the handout gestures at without requiring.

What it overlooked was a table that cannot answer the question it exists for. Claude built exactly what the handout asked: ten rows, six columns, three models. It did not flag that the resulting Task-2 table would contain sixty red cells and no black ones, and that such a table cannot support deliverables (3) or (4). A table with zero variance carries no information about ranking. I had to notice that and extend the model set. The assistant optimised for compliance with the specification rather than for the purpose the specification serves.

It also miscalculated the token budget. `num_predict` was initially 640 for Task-2 CoT. At that budget the 1.5 B model truncated 5 of 10 long divisions mid-computation. A truncated response is a harness defect, not a model error, and scoring those five as wrong would have attributed my configuration mistake to the model. I found this by reading raw responses, not by looking at accuracy. The budget is now keyed by `(task, arm)` and set to 1536 for Task-2 CoT, and it is declared in the report as a stated deviation rather than quietly applied.

The "modal answer" convention also hides disagreement. Each cell reports the modal normalised answer over k draws, so where the draws disagree the cell shows the plurality and the spread disappears. For Table 1 this matters. Gemma-3-4B's two correct cells are correct at every draw, but several red cells are red at different wrong strings on different draws. The HTML build carries the draw count in the cell's `title` attribute; the printed table does not, so a reader of the PDF cannot tell a unanimous wrong answer from a 2-of-3 one. The self-consistency column in the report's summary section reduces the problem without fixing it.

One limitation I accept: ten items per task means one item is worth 10% of a column. As Question 3 shows, two strings account for Gemma-3-4B's entire reversal score. The tables are honest but small, and no amount of prompting changes that.

Question 3 (Deliverable 3): compare the 10 queries and the results across all three pretrained LLMs, with observations for each

Step 1: Final answer (AI generated)

Across 14,935 graded trials (7 models, 2 tasks, up to 5 prompting arms, 11 temperatures, k draws) four observations hold.

Division has a sharp size threshold; reversal does not

The three models at or below 7.25 B, meaning Qwen2.5-1.5B, Gemma-3-4B and Mistral-7B, score 0.000 on division across 2,480 trials spanning every temperature and both non-verbose arms. Above that point accuracy rises monotonically: 0.103 (Qwen2.5-7B, 7.62 B), 0.048 (Llama-3.1-8B), 0.300 (Gemma-3-12B), 0.824 (Qwen3-32B). The threshold sits between 7.25 B and 7.62 B, inside the interval the required three-model ladder leaves unsampled.

Reversal shows no such ordering. Zero-shot accuracy by size runs 1.54 B at 0.000, 4.30 B at 0.198, 7.25 B at 0.000, 7.62 B at 0.061, 8.03 B at 0.048, 12.2 B at 0.185, 32.8 B at 0.381. Gemma-3-4B beats both 7 B models and the 8 B model. Qwen2.5-1.5B and Mistral-7B both score zero despite a 5.7 B parameter gap. Only the 32.8 B model is clearly ahead, and even its best reversal score, 0.500 with CoT, is far below its division score.

Temperature is nearly irrelevant; the item is what matters

The pooled accuracy curve over the 11 temperatures has a total range of 0.03. The per-item breakdown explains why: outcomes are almost always constant across the whole sweep. For Qwen3-32B on division, 7 of 10 items are correct at all eleven temperatures, 1 is wrong at all eleven, and only 2 are mixed. An aggregate temperature sweep on its own would report "temperature does not matter" and miss that the cause is item-level determinism rather than insensitivity to sampling.

The exceptions are models sitting just above zero. Llama-3.1-8B's reversal accuracy decays from its greedy value towards zero as temperature rises, because sampling walks it off an answer it can occasionally reach. Models that fail consistently are unaffected, since there is nothing to walk away from.

Per-item concentration

Gemma-3-4B's reversal score of 0.198 comes from 1 string it always gets right (JQJPc) and 1 it usually gets right (gypHq). The other 8 are never correct across 550 trials. Both winners are 5-character strings, and JQJPc has only 4 distinct characters. That is not evidence of reversal ability, it is evidence that two easy strings entered the sample. Only Qwen3-32B, with 2 always-correct and 3 sometimes-correct items, succeeds broadly enough to suggest a general capability.

Length hurts, but unevenly

Both Gemma models collapse from the 5-character band to the 8-character band: the 4 B goes 0.396 to 0.000 and the 12 B goes 0.352 to 0.018. Five characters is apparently near their limit. But Qwen2.5-7B and Llama-3.1-8B score higher on the longer band, and Qwen3-32B is essentially flat. With five items per band, item difficulty and length cannot be separated.

Per-model notes

Qwen2.5-1.5B (1.54 B) scores zero on both tasks zero-shot. Its reversal failures are dominated by wrong length and wrong characters: it produces plausible-looking strings that are not permutations of the input, turning `PkUuq` into `rqsw` and introducing `r`, `s` and `w`. It is the only sub-threshold model whose reversal responds to few-shot, going from 0.000 to 0.068.

Gemma-3-4B (4.30 B) has the best sub-threshold reversal at 0.198, driven by two items, and zero division. Its most striking behaviour appears under CoT, discussed in Question 5.

Mistral-7B (7.25 B) scores zero on both tasks zero-shot despite being the largest of the required three. Its division outputs at $t=1.0$ frequently collapse into echoing the numerator (`744`, `944`, `159`, `258`, `81`): it stops solving and starts copying. Few-shot recovers reversal to 0.100, the best of the three-model ladder in any arm.

Qwen2.5-7B (7.62 B) is the first model above the division threshold at 0.103, and few-shot doubles it to 0.212. It commits 19 pure rounding errors, meaning a correct quotient with a wrong final digit. That failure mode does not exist below the threshold, because below it the quotient itself is wrong.

Llama-3.1-8B (8.03 B) is weak on both tasks (0.048 and 0.048) and uniquely fragile to format. The stop-sequence bug in Question 5 affected 235 of 330 trials in one of its cells.

Gemma-3-12B (12.2 B) scores 0.185 and 0.300. It is the only model whose dominant reversal failure is wrong order (112 instances) rather than wrong characters: it keeps the multiset and misplaces it, which is a qualitatively closer failure.

Qwen3-32B (32.8 B) scores 0.381 and 0.824, best on everything. It runs with reasoning disabled so its zero-shot arm measures the same thing as the others'.

Step 2: AI interaction record

1) Tool URL. As per the shared block.

2) Exact prompts used.

```
why is mistral behaving worse than the smaller param models?
why is mistral not able to asnwer even thought it is a significantly largert model? also stop the COT runs all together, we
will only run zero and few shot ones
do you think we should run larger models to see if they are better?
create an md file explaining the prooject structure and how the results are stored. I want to analyze the data with a
different agent
is it a good idea to run smaller models with few shot and COT since they have very little context window
this project started as a hw, but I want to use this as a standalone project. I want to keep this run explorer, but I want
to document the PS, my assumptions and analysis and conclusions from the run data into the UI itself. make changes to the UI
for that, but keep the explorer UI more or less the same
```

3) Multi-round workflow.

Round	Context	Query	Outcome
1	Mistral scoring below the 1.5 B model	why is Mistral worse than smaller models?	Claude's first answer was a capability story. I did not accept it.
2	Same, restated	why can't it answer even though it is much larger?	The second look found a harness cause: Mistral kept generating past its answer and invented its own few-shot examples, so 52% of one cell hit the 128-token cap. Fixed with a <code>["\n"]</code> stop sequence.
3	Three-model results	should we run larger models?	Yes, which led to the four supplementary models.

Round	Context	Query	Outcome
4	Full run corpus	write a doc describing structure and result schemas, I want a different agent to analyse the data	H1-data-guide.md. A deliberate independent-verification step.
5	Context-window worry	is few-shot or CoT sensible for small models with small context?	Confirmed 3-shot verbose fits in num_ctx 2048 for every model, so the arms are comparable.
6	Complete corpus	put the problem statement, assumptions, analysis and conclusions into the UI itself	src/writeup.py: authored prose with {acc qwen3-32b task1 cot}-style placeholders resolved from runs/ at build time.

4a) Prompting strategy and multi-round AI-use strategy.

Three techniques did the work here.

The first was re-asking a question whose answer sounded plausible. Round 1's answer, that Mistral is weaker at character manipulation, was fluent, consistent with the numbers, and wrong about the mechanism. Asking the same question again with the anomaly restated more sharply ("even though it is a significantly larger model") pushed Claude from explaining the number to investigating it, and the second pass found a truncation bug in my own harness. When an assistant explains an anomaly without inspecting raw data, the explanation is a hypothesis wearing the clothes of a finding.

The second was handing the data to a second, uncontaminated reader. Round 4's prompt asks for a schema document explicitly so that a different agent could analyse the runs. An assistant that wrote the harness has an interest in the harness being correct; a reader that only sees `runs/*.jsonl` and a schema does not. This is cheap adversarial review, and it shares its motivation with computing Task-2 gold by two independent methods.

The third was forcing the prose to be a function of the data rather than a snapshot of it. Round 6 is the strongest anti-hallucination control in the project. Every number in the analysis is a placeholder resolved from the committed traces when the page is built, and an unresolvable reference raises rather than rendering blank. A hand-typed results paragraph goes stale the instant a cell is re-graded, and a stale paragraph in an AI-assisted report is indistinguishable from a fabricated one. This makes staleness impossible by construction instead of something I have to remember to check.

4b) Critique of the AI's response.

The item-level stability analysis is the best thing the assistant produced. Given the pooled curve, the obvious write-up is "temperature has no effect". Claude instead decomposed the sweep per item and showed why: outcomes are item-determined. That turns a null result into a substantive one, and it is a better analysis than I would have written unaided.

On the Mistral question, round 1's answer was confidently wrong. This is the most important critique in this document. The AI's default failure mode is to explain data rather than doubt the instrument. It had access to the raw responses the whole time, and it reached for a story about model capability because that is the genre of answer the question invited. Only an explicitly incredulous re-prompt moved it to check the pipeline.

Early drafts also overstated causation. They said things like "Gemma-3-4B is better at short strings". With 5 items per band and a score built on 2 strings, that sentence is not supported. The current text says these are observations from the current sample rather than evidence of a general length effect, and calls for an order of magnitude more items. That hedging was added at my insistence rather than volunteered. An assistant asked to write findings will write findings; it will not spontaneously write "this is underpowered".

One confound the assistant did not raise until asked: the seven models span four organisations and two model generations, all at Q4_K_M. Only Qwen2.5-1.5B to Qwen2.5-7B is a clean within-family size comparison. Everything else varies pretraining corpus, training recipe and release date alongside size, so the threshold between 7.25 B and 7.62 B is a threshold in this set of checkpoints rather than a scaling law. Quantisation is a further uncontrolled variable that plausibly bears on character-level tasks specifically, and it was not varied. These are in the Limitations section now.

My synthesis: the defensible claims are that division shows a sharp capability threshold in this checkpoint set with a qualitative change in error type at the boundary (quotient errors below, rounding errors above); that reversal is not size-ordered below 32 B and its errors are predominantly character-destroying rather than order-scrambling, consistent with subword tokenisation; and that temperature is not a useful lever for either task between 0.0 and 1.0. All three rest on ten items per task and should be treated as directional.

Question 4 (Deliverable 4): for each LLM, compare the two learning tasks and the in-context phrases designed, with pros, cons and behaviour across the two complexity bands

Step 1: Final answer (AI generated)

The in-context phrases I designed

Five prompt arms, frozen before the first run and version-stamped (`PROMPT_VERSION = "v2"`). Any edit bumps the version.

Arm	Task-1 phrasing (abridged)	Task-2 phrasing (abridged)
zeroshot	"Reverse the following string. Output only the reversed string, nothing else."	"Divide the first number by the second number and round the result to {places} decimal places. Output only the final number, nothing else."
fewshot	Same instruction plus 3 held-out String: <code>s -> gold</code> pairs from the item's own length band	Same instruction plus 3 held-out Numbers: <code>a, b -> gold</code> pairs from the item's own precision band
cot	"...Work through it one character at a time, listing the characters from last to first. Then give the final answer on a line beginning \"FINAL: \"."	"...Work through the long division step by step, one digit at a time. Then give the final answer on a line beginning \"FINAL: \"."
fewshot_verbose	5 numbered rules (read right to left, copy every character once, preserve case, equal length, output only the string) plus worked examples showing the characters spaced out	5 numbered rules (do not reverse operands, quotient before rounding, exactly {places} digits, preserve trailing zeros, output only the number) plus examples showing the operation explicitly
cot_verbose	A required output format: INDEX:, REVERSED INDEXES:, CANDIDATE:, CHECK: input length=l, output length=..., FINAL:	A required long-division schedule: INTEGER:, then per digit <code>x = 10*remainder, digit = x // B, product, new remainder</code> , with two verification inequalities, plus GUARD: digit, ROUNDED:, FINAL:

Few-shot examples are drawn from a pool generated with a different seed (916220 against 6220), asserted disjoint from the test set, and taken from the item's own complexity band. The technique is therefore never confounded with a difficulty mismatch, and an example can never supply the answer.

Accuracy by model, task and arm

Normalised, all temperatures pooled, n in parentheses.

Task 1, reversal:

Model	zero-shot	few-shot	few-shot verbose	CoT
qwen2.5-1.5b	0.000 (550)	0.068 (546)	n/a	0.032 (493)
gemma3-4b	0.198 (550)	0.000 (550)	n/a	0.000 (330)
mistral-7b	0.000 (550)	0.100 (549)	n/a	0.003 (328)
qwen2.5-7b	0.061 (329)	0.040 (329)	n/a	0.079 (329)
llama3.1-8b	0.048 (330)	0.000 (95)	n/a	0.077 (324)
gemma3-12b	0.185 (330)	0.112 (330)	0.224 (330)	0.088 (330)
qwen3-32b	0.381 (328)	0.370 (330)	0.102 (283)	0.500 (330)

Task 2, division:

Model	zero-shot	few-shot	few-shot verbose	CoT
qwen2.5-1.5b	0.000 (549)	0.000 (550)	n/a	0.000 (48)
gemma3-4b	0.000 (330)	0.000 (329)	n/a	n/a
mistral-7b	0.000 (329)	0.000 (327)	n/a	n/a
qwen2.5-7b	0.103 (330)	0.212 (330)	n/a	n/a
llama3.1-8b	0.048 (330)	0.025 (321)	n/a	n/a
gemma3-12b	0.300 (330)	0.200 (330)	0.200 (330)	n/a
qwen3-32b	0.824 (330)	0.830 (330)	0.700 (330)	0.722 (18)*

* Qwen3-32B Task-2 CoT is provisional. The SSH tunnel to the GPU node dropped mid-run and only 18 valid trials were collected.

The two tasks are not the same kind of hard

Division is arithmetic-hard: it has a capability threshold. Below 7.25 B nothing works at any temperature under any prompt, and above 7.62 B accuracy climbs steeply to 0.824. Reversal is representation-hard: it resists size across a 20x parameter range, and the errors say why. Most reversal failures are wrong length or wrong chars , so the output is not a permutation of the input at all. The model produces a string-shaped thing instead of rearranging the given characters, which is what you would expect if the input reaches it as a handful of subword tokens rather than a sequence of characters. Division errors are overwhelmingly wrong digits : 99% of Qwen2.5-1.5B's and 100% of Mistral-7B's division errors are genuinely wrong quotients rather than formatting problems. Strict and normalised grades are identical in every zero-shot and few-shot division cell, which confirms presentation is not the issue there.

Prompting helps some models and hurts others, by large margins

CoT produces the single largest gain in the study (Qwen3-32B reversal, 0.381 to 0.500, a change of +0.119) and also the single largest loss (Gemma-3-4B reversal, 0.198 to 0.000). Few-shot doubles Qwen2.5-7B's division (0.103 to 0.212) and cuts Gemma-3-12B's by a third (0.300 to 0.200). The verbose few-shot rewrite is the only arm in which Gemma-3-12B beats its own zero-shot reversal, at 0.224, and the same rewrite drops Qwen3-32B from 0.370 to 0.102. Examples, seeds, sampling settings and token budget are identical between fewshot and fewshot_verbose ; only the wording changes. The prompt-technique effects are larger than the differences between models, which means any claim of the form "CoT improves task X by N%" is meaningless without naming the model.

Complexity bands

Reversal from l = 5 to l = 8 destroys both Gemma models (0.396 to 0.000, and 0.352 to 0.018), helps Qwen2.5-7B and Llama-3.1-8B, and leaves Qwen3-32B flat. Division from 5 dp to 8 dp is the more interesting boundary because it is where the error type changes: Qwen2.5-7B's 19 rounding errors are concentrated where the guard digit matters.

Per-model pros and cons

Model	Pro	Con
Qwen2.5-1.5B	Format-obedient, responds to few-shot on reversal (0 to 0.068)	No arithmetic at all, invents characters not in the input
Gemma-3-4B	Best sub-threshold reversal, perfectly stable on the items it knows	CoT destroys it (see Q5), zero division, collapses at l = 8
Mistral-7B	Best three-model-ladder reversal under few-shot (0.100)	Largest of the required three and zero-shot zero on both tasks, degenerates into echoing the numerator at t = 1.0
Qwen2.5-7B	First model that can divide, benefits from every arm	Rounding step is unreliable (19 pure rounding errors)
Llama-3.1-8B	Gains most from CoT relative to its zero-shot (0.048 to 0.077)	Extremely format-fragile, the only model the stop-sequence bug destroyed
Gemma-3-12B	Best sub-32 B division (0.300), error type is wrong order and therefore closest to right	Prompting mostly hurts it, verbose is the only arm that helps
Qwen3-32B	Best everywhere, CoT actually works on reversal	Verbose prompting halves its few-shot score, reasoning had to be disabled for comparability

Step 2: AI interaction record

1) Tool URL. As per the shared block.

2) Exact prompts used.

```
after this can you also run few shot and CoT prompt for qwen
cot and few shot should also have runs for all the temps na
Work in /Users/atharvamohite/dev/acads/bds/HW1. Make a small, focused change. Do not externalize or rewrite the prompt
system yet. Add two new prompt arms so the project has five: zeroshot, fewshot, cot, fewshot_verbose, cot_verbose. Use the
correct spelling cot_verbose.
please stop the CoT runs. I want to run zeroshot for each model first, then few shot and only then CoT
lets run CoT (not the verbose one) for the ones that don't have a run for it yet go ascending
lets do few shot verbose for task1 and task2 for 12b and 32b only
can we not run it in parallel?
```

3) Multi-round workflow.

Round	Context	Query	Outcome
1	Zero-shot grid complete	also run few-shot and CoT	Arms extended.
2	Arms only at endpoints	CoT and few-shot should also cover all temperatures	Full 11-point grid per arm, so arm and temperature are crossed rather than confounded.
3	Terse arms underperforming	add fewshot_verbose and cot_verbose, do not rewrite the prompt system, keep the originals byte-identical	Separates "the technique doesn't help" from "my instruction was too terse". The terse arms become the control.
4	Mixed run ordering	run zero-shot for every model, then few-shot, then CoT	Re-ordered arm-outermost so a partial corpus stays balanced across models.
5	Missing CoT cells	run CoT (not verbose) for models lacking it, ascending by size	Filled the grid cheapest first.
6	GPU time limited	verbose arms for 12 B and 32 B only	Targeted the models where wording plausibly matters.
7	Claude serialised two jobs	can we not run it in parallel?	A correction. A shared GPU with concurrent decode makes timings and memory pressure non-comparable.

4a) Prompting strategy and multi-round AI-use strategy.

Round 3 is the design decision I am most pleased with, and it came from a diagnostic worry rather than a plan. The terse CoT and few-shot arms were underperforming zero-shot on several models. There are two possible explanations, that the technique does not help these models or that my instruction was too vague, and a single arm cannot separate them. So I added verbose variants that use the same technique with rules and output format spelled out, and I explicitly told Claude not to touch the existing prompts: "Do not externalize or rewrite the prompt system yet." Keeping the terse originals byte-identical turns them into a control condition. Without that instruction an assistant will improve the prompts it is extending, and the comparison evaporates.

Round 2 applies the same instinct to the sweep. Arms had initially been run only at the endpoints, which would have confounded prompting arm with temperature. Insisting on the full grid per arm costs GPU hours and buys the ability to say anything at all about interaction.

Round 7 is a correction of the assistant. Claude serialised a CoT job and a Task-2 job to be safe. On a shared A100 with one Ollama process, concurrent decode changes both timings and memory pressure, and the trace header records a backend label precisely because those two things are not comparable across configurations. I overruled it.

Sequencing in rounds 4, 5 and 6 was also deliberate. Arm-outermost and ascending by size means that running out of cluster time leaves a balanced partial corpus rather than a complete picture of two models and nothing about the rest. That turned out to matter: the Qwen3-32B Task-2 CoT cell is the incomplete one, and it is the last cell in the ordering.

4b) Critique of the AI's response.

The five-arm design, the band-matched shot selection, and the disjointness assertion between pool and test set are all correctly implemented, and the per- (task, arm) token budget correctly recognises that eight-place long division needs far more room than listing eight characters backwards.

Claude's initial framing treated CoT as a technique with a direction, roughly "CoT improves reasoning". The data contradicts this: the largest gain and the largest loss in the entire study are both CoT on the same task. For Gemma-3-4B, CoT takes reversal from 0.198 to zero, not to a lower number but to zero. Any general statement about prompting techniques, of the kind that appears constantly in both papers and assistant output, is unsupported at this sample size. The write-up now says so.

It also missed a confound in its own design. When fewshot_verbose produced a large drop for Qwen3-32B, from 0.370 to 0.102, the drafted explanation was about prompt wording. Part of it was truncation: that cell loses 47 of 330 trials, and until the grading fix those losses were being counted as wrong answers. The assistant proposed a semantic explanation for what was partly an instrumentation artefact. The effect survives the fix, but it is smaller, and the first explanation was overconfident.

The stated deviations were under-specified at first. num_predict and stop vary by (task, arm) . That is necessary, since a single-line answer and a 1536-token long division cannot share a budget, but it means the arms are not identical in every respect, and the first draft did not say so. They are declared in the report's assumptions now.

One weakness I accept: the cot_verbose arm was written and frozen but never run at scale. GPU time went to fewshot_verbose on the two largest models instead. Its column is absent from the tables above rather than empty, and I would rather report that than pad the grid.

Question 5 (Deliverable 5): critique of the LLM reasoning outputs used for analysis

Step 1: Final answer (AI generated)

This deliverable has two halves, because two different populations of AI reasoning were involved. Both need critiquing.

Part A: critique of the subject models' reasoning (the CoT traces)

A1. Correct reasoning, wrong answer. Gemma-3-4B, Task 1, cot arm, t = 0.0, Q1 (PkUuq , gold quUkP). Verbatim:

```
Okay, let's reverse the string "PkUuq" one character at a time:
```

1. q
2. u
3. U
4. k
5. P

```
FINAL: PkUuq
```

The enumeration is completely correct. q, u, U, k, P is exactly the answer. The model then writes the input after FINAL: . This is a failure to carry a correct intermediate result into the answer slot, and it happens on Q2 identically: inNj f produces the correct listing f, j, N, n, i and then FINAL: inNj f . It is the mechanism behind Gemma-3-4B's CoT score of 0.000. The model that scores highest on sub-threshold reversal zero-shot scores zero when asked to show its work, while demonstrably knowing the answer. Any evaluation that reads only the final line records this as "cannot reverse strings", which is the opposite of what the trace shows.

A2. Correct answer, wrong reasoning. Qwen3-32B, Task 2, cot arm, t = 0.0, Q1 (744 / 336). Verbatim excerpt:

We are dividing 744 / 336.
Since 336 is larger than 744, the result will be greater than 1 but less than 2.

336 is not larger than 744. The stated premise is false, the conclusion drawn from it is correct, the long division that follows is correct, and the final answer 2.21429 is correct. The verbal reasoning here is narration alongside the computation rather than the computation itself, and it can be wrong without the answer being wrong.

A3. What A1 and A2 jointly establish. A CoT trace is not an audit trail. In A1 correct reasoning coexists with a wrong answer; in A2 incorrect reasoning coexists with a right answer. Neither direction of inference holds. A grader who trusts the reasoning text to explain the answer will be wrong in both cases. This is the locally observed version of the published finding that chain-of-thought explanations are frequently unfaithful to the process that actually produced the answer (Turpin et al., 2023; Lanham et al., 2023), and it is why the grading in this project reads only the **FINAL:** line and why the raw traces are published in the explorer rather than summarised.

A4. Degenerate reasoning under sampling. Mistral-7B at $t = 1.0$ on division stops computing and echoes the numerator (744 , 944 , 159 , 258 , 81). The output is syntactically an answer and semantically a copy of the input.

Part B: critique of the assistant model's reasoning, and the harness bug it caused

The most important finding in this project about AI-generated analysis is that an evaluation artefact was, for a period, being reported as a model weakness.

The non-CoT arms instruct "output only the answer, nothing else", so generation stops at the first newline (**STOP = ["\n"]**). Llama-3.1-8B does not comply. It begins few-shot reversal responses with a preamble:

Here are the reversed strings:

Generation then stops at the newline, before the answer exists. The stored response is that sentence and nothing else, and the grader scored it as a wrong answer. This affected 235 of 330 trials in that cell, so Llama's few-shot reversal was being reported as 0.000 on the basis of a stop sequence rather than a model failure. A similar pattern appeared in Qwen3-32B's verbose few-shot reversal.

I caught it because the size of the drop was implausible. The **fewshot** and **fewshot_verbose** cells shared examples, seeds, sampling settings and token budget, yet accuracy fell from 0.370 to 0.102. A wording change should not do that.

The fix classifies responses that become empty after normalisation as truncated, exactly like CoT responses missing a **FINAL:** marker, and excludes them from the denominator rather than counting them as wrong. The existing raw responses were re-graded from stored text rather than re-run, because re-running would have drawn different samples and made before and after incomparable. Of 14,916 re-graded trials, 312 were reclassified as invalid, with 0 changes to any correctness verdict. The fix changes which trials count; it never turns a wrong answer into a right one.

Scoring a truncated response as wrong mixes my evaluation failure with the model's weakness, and the mixture is invisible in aggregate accuracy because it looks exactly like a model that cannot do the task. The only thing that distinguished them was reading the raw text. The assistant that wrote the stop-sequence logic also wrote the analysis of the resulting numbers, and it explained those numbers as model behaviour without suspecting its own instrument. That is a structural conflict of interest in AI-assisted evaluation rather than a one-off bug.

Step 2: AI interaction record

1) **Tool URL.** As per the shared block. The traces critiqued in Part A were produced by **gemma3:4b** , **qwen3:32b** and **mistral:7b** on Ollama. The reasoning critiqued in Part B is Claude's.

2) **Exact prompts used.**

```

there is no data in the CoT and few shot section of the UI. there seems to be a wiring issue
why is mistral behaving worse than the smaller param models?
make it work like the other models
run CoT for task2 just for 32b first and then rerun the grader bug one
is the change for regex matching in case of thinking made now?
remove the normalized and strict grading toggle, just use the normalized value

```

Followed by an explicit choice, offered by the assistant as options and decided by me: "Fix `grade.py`, re-grade from stored raw", scope "all affected cells".

3) Multi-round workflow.

Round	Context	Query	Outcome
1	Empty CoT and few-shot panels	there's a wiring issue in the UI	A UI bug, not a data bug. Cleared the ground.
2	Mistral below the 1.5 B model	why is Mistral behaving worse than smaller models?	First answer was a capability story. Not accepted.
3	Same anomaly	make it work like the other models	Found the runaway continuation. Mistral wrote its own few-shot examples past the answer, truncating 52% of one cell at 128 tokens. Introduced STOP .
4	Qwen3-32B fewshot against fewshot_verbose gap	run Task-2 CoT for 32 B, then rerun the grader bug one	I named it a grader bug before the assistant did.
5	Options presented	fix <code>grade.py</code> and re-grade from stored raw, all affected cells	14,916 re-graded, 312 reclassified, 0 verdict changes.
6	Thinking-model handling	is the regex change for thinking-model output made yet?	Verified Qwen3's think-block handling would not be truncated into an empty answer.
7	Report surface	drop the strict/normalised toggle	Single headline metric, strict retained internally.

4a) Prompting strategy and multi-round AI-use strategy.

The strategy across these rounds was to treat an implausible number as an instrument fault until proven otherwise. Three implausible numbers appeared: Mistral below a model a fifth its size, Llama's few-shot cell at zero, and a 0.27 accuracy swing from a pure wording change. All three were instrument faults, wholly or partly. None was found by looking at accuracy. All three were found by reading raw responses.

Round 4's phrasing matters. By the time I wrote "rerun the grader bug one" I had already decided it was a grading problem, and naming it constrained the assistant to investigating the grader rather than re-explaining the model. Under-constrained prompts get plausible narratives; a prompt that names the suspect component gets an investigation.

Round 5's decision to re-grade from stored raw rather than re-run is what keeps the before and after comparison meaningful. Re-running would have drawn different samples, so any change would confound the fix with sampling noise. That the re-grade produced 0 changes to correctness verdicts is the audit trail proving the fix only moved trials between counted and excluded; it did not manufacture accuracy.

Publishing raw responses in the explorer, truncated to 600 characters each so a full CoT sweep does not produce a 40 MB page, is a deliberate anti-hallucination measure for the reader. Any number in the write-up can be traced to the individual draws behind it.

4b) Critique of the AI's response, and my final refined synthesis.

Given the correct suspicion, the assistant's diagnosis was fast and complete. It found the stop-sequence interaction, correctly identified that a marker keyed to the prompt's own label (`"\nString:"`) had already been tried and leaked, and verified that no response among the 8,137 collected before the change began with a newline, so the fix could not truncate a real answer to empty. It also proposed re-grading from raw over re-running, which was the right call. That is careful work.

What it got wrong follows a pattern. Every one of the three instrument faults was first explained as model behaviour. This is not random error. I think the cause is that the assistant is asked "why is X low?" and answers the question as posed, inside the frame it is given, when the frame itself, that the number is a measurement, is what should have been questioned. The AI is a good debugger and a poor sceptic. It will investigate thoroughly once told where to look, and it will rarely tell you that the place to look is its own code.

Draft prose also reached repeatedly for tidy causal language: "CoT improves reasoning", "larger models are better at arithmetic", "Gemma is better at short strings". Each is contradicted or unsupported by this data. CoT has the largest gain and the largest loss in the study, the size relationship exists for one task and not the other, and Gemma's short-string advantage rests on two items out of five. The hedges in the current write-up were added by me.

Two miscalculations are worth naming: the Task-2 CoT token budget of 640, which truncated 5 of 10 long divisions on the 1.5 B model, and the initial treatment of truncated responses as wrong answers. Both are the same class of error, a configuration choice whose failure mode is silent and looks exactly like poor model performance.

Three claims survive scrutiny. First, a chain-of-thought trace is evidence about neither the answer nor the process. A1 and A2 above are counterexamples in opposite directions, from two different models, in the same corpus. Grading must read the answer slot and nothing else, and reasoning text should be published rather than trusted. Second, in AI-assisted evaluation the assistant that builds the instrument cannot be relied on to doubt it. Concretely, 235 of 330 trials in one cell were reported as model failures caused by my own stop sequence, and the aggregate metric could not distinguish them. Independent controls are what made the error findable: ground truth computed twice by different methods, a schema document so another agent could read the runs cold, prose whose numbers are resolved from the traces at build time, and re-grading from stored raw with a verdict-change count of zero. Third, the most valuable action in the entire project was reading raw model output. Every real finding here, the `FINAL:` mismatch, the false premise inside a correct derivation, the numerator echo, and the preamble truncation, is invisible in accuracy and obvious in the text. An AI-assisted analysis that never opens the raw data is a summary of a summary.

Step 3: References

Handout-provided sources:

1. Ravaut, M., Ding, B., Jiao, F., Chen, H., Li, X., Zhao, R., Qin, C., Xiong, C., & Joty, S. (2024). *A Comprehensive Survey of Contamination Detection Methods in Large Language Models*. arXiv:2404.00699.
2. Cheng, Y., Wang, Y., Liu, Y., et al. (2024). *Unveiling the Spectrum of Data Contamination in Language Models: A Survey from Detection to Remediation*. Findings of the Association for Computational Linguistics: ACL 2024.

Prompting and reasoning:

1. Brown, T. B., Mann, B., Ryder, N., et al. (2020). *Language Models are Few-Shot Learners*. Advances in Neural Information Processing Systems 33 (NeurIPS 2020). arXiv:2005.14165.
2. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022. arXiv:2201.11903.
3. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. ICLR 2023. arXiv:2203.11171.
4. Turpin, M., Michael, J., Perez, E., & Bowman, S. R. (2023). *Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting*. NeurIPS 2023. arXiv:2305.04388.
5. Lanham, T., Chen, A., Radhakrishnan, A., et al. (2023). *Measuring Faithfulness in Chain-of-Thought Reasoning*. arXiv:2307.13702.
6. Sclar, M., Choi, Y., Tsvetkov, Y., & Suhr, A. (2024). *Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design*. ICLR 2024. arXiv:2310.11324.

Decoding, tokenisation, scale and quantisation:

1. Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). *The Curious Case of Neural Text Degeneration*. ICLR 2020. arXiv:1904.09751.
2. Sennrich, R., Haddow, B., & Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. ACL 2016. arXiv:1508.07909.
3. Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). *Training Compute-Optimal Large Language Models*. NeurIPS 2022. arXiv:2203.15556.
4. Dettmers, T., & Zettlemoyer, L. (2023). *The Case for 4-bit Precision: k-bit Inference Scaling Laws*. ICML 2023. arXiv:2212.09720.

Model documentation for the seven checkpoints under test:

1. Qwen Team, Alibaba Cloud (2024). *Qwen2.5 Technical Report*. arXiv:2412.15115.
2. Qwen Team, Alibaba Cloud (2025). *Qwen3 Technical Report*. arXiv:2505.09388.
3. Gemma Team, Google DeepMind (2025). *Gemma 3 Technical Report*. arXiv:2503.19786.
4. Jiang, A. Q., Sablayrolles, A., Mensch, A., et al. (2023). *Mistral 7B*. arXiv:2310.06825.
5. Grattafiori, A., Dubey, A., Jauhri, A., et al. (2024). *The Llama 3 Herd of Models*. arXiv:2407.21783.

Tooling and infrastructure:

1. Ollama (2026). *Ollama documentation and model library*. <https://ollama.com>, API reference at <https://github.com/ollama/ollama/blob/main/docs/api.md>
2. Anthropic (2026). *Claude Code*. <https://claude.ai/code>, API reference at <https://docs.anthropic.com>
3. Partnership for an Advanced Computing Environment (PACE), Georgia Institute of Technology. *PACE ICE cluster documentation*. <https://docs.pace.gatech.edu>

Note: the arXiv identifiers should be spot-checked against the arXiv listing before final submission.

Step 4: Appendices

A. Project repository and live artefacts. The interactive page is live at <https://llm-prompting-lab.vercel.app/>. It has two panes over one corpus: the analysis write-up, and a trial explorer where any query by temperature cell opens the raw responses behind it. Everything builds to a single self-contained file with no server and no CDN, and the numbers in the write-up pane are resolved from the committed run files at build time. Source is at <https://github.com/moriowen/llm-prompting-lab> (private).

B. Reproduction.

```
python3 -m src.main generate # rebuild data/*.json from the committed seeds
python3 -m src.main verify   # assert the ground-truth invariant
python3 -m src.main models   # capture digests + per-model default hyperparameters
python3 -m src.main run --models qwen2.5-1.5b,gemma3-4b --temps grid --k 5
python3 -m src.main sweep    # apply the pre-committed selection rule
python3 -m src.main report    # -> report.html (refuses to build if verify fails)
python3 -m src.main ui        # -> ui.html (write-up + explorer)
```

Zero pip dependencies: `urllib` for HTTP, `decimal` and `fractions` for arithmetic. Runs are committed JSONL, one file per (model, task, arm, temperature), flushed per line, and every run is resumable because `run` skips any (qid, seed) already recorded.

C. Corpus size. 14,935 graded trials: 7 models, 2 tasks, up to 5 arms, 11 temperatures, k draws (k = 3 across the grid, topped up to k = 10 at the two reported temperatures).

D. Data schema. `H1-data-guide.md` in the repository documents the JSONL trace format: one `run` header carrying the frozen configuration, then one `trial` line per call recording prompt, raw response, extracted answer, normalised

answer, strict and normalised verdicts, error type, `done_reason`, prompt and output token counts, wall time, and backend label.

E. The full AI interaction transcript. The complete Claude Code session history for this project, six sessions between 2 and 4 September 2026 covering 139 user turns, is available as JSONL and can be supplied on request. The prompts quoted throughout Step 2 are verbatim excerpts from it, including my own typos, which I have not corrected.

F. Stated deviations from a fully controlled design. `num_predict` varies by `(task, arm)`. `stop` varies by `(task, arm)`. Qwen3-32B runs with reasoning disabled. Qwen3-32B Task-2 CoT is incomplete at 18 valid trials because the SSH tunnel dropped. `cot_verbose` was frozen but never run at scale. Two backends were used, the laptop and the A100, and accuracy is compared across them while wall-clock timings are not. Five of the seven models are pinned by tag rather than by locally captured digest.